

Model Training using Ray Framework

Prerequisites

- **Hardware**:
 - Minimum: 8 GB RAM, quad-core CPU, 50 GB free disk space per laptop.
 - Recommended: 16 GB RAM, 8-core CPU, SSD with 100 GB free space.
- **Network**: Both laptops on the same local network with static IPs (e.g., Laptop 1: `192.168.1.100`, Laptop 2: `192.168.1.101`).
- **OS**: Ubuntu 24.04 (Linux-based). Adjust for macOS/Windows if needed.
- **Dataset**: Stored on Laptop 1 (e.g., `/home/user/data/dataset.csv`) or shared storage.
- **Tools**: Python, PyTorch, Ray framework and networking utilities.

Overview

- **Architecture**:
 - **Laptop 1** (e.g., IP: `192.168.1.100`): Parameter server (rank 0) and worker.
 - **Laptop 2** (e.g., IP: `192.168.1.101`): Worker (rank 1).
 - The parameter server aggregates gradients and updates model parameters, while workers process data subsets.
- **Dataset**: Assume a huge dataset (e.g., CSV or Parquet) is split and stored locally on each laptop to avoid network bottlenecks.
- **Framework**: PyTorch with Gloo for distributed training.
- **Goal**: Train a neural network on a large dataset using a RAY framework approach.

Prerequisites (Cross check for dataset partition)

- **Hardware** (per laptop):
 - Minimum: 8 GB RAM, quad-core CPU, 50 GB free disk space.
 - Recommended: 16 GB RAM, 8-core CPU, SSD with 100 GB free space.
- **Network**: Both laptops on the same local network with static IPs.
- **Operating System**: Ubuntu 24.04 (Linux-based). Adjust for macOS/Windows if needed.
- **Dataset**: A huge dataset (e.g., `/home/user/data/dataset.csv`) split across laptops.
- **Tools**: Python, PyTorch, and networking utilities.

1. Ray Cluster

Description: Ray is a distributed computing framework for Python, designed for ML and reinforcement learning. It supports distributed training, hyperparameter tuning, and data processing, making it a lightweight alternative to Spark for ML workflows.

Setup Steps:

1. **Install Ray**:

```
```bash
pip3 install ray torch pandas
```
```

2. ****Start Ray Head Node (Laptop 1)**:**

```
```bash
ray start --head --port 6379 --redis-password='password'
```
```

Note the address: `ray://192.168.1.100:6379`.

3. ****Join Worker Node (Laptop 2)**:**

```
```bash
ray start --address='192.168.1.100:6379' --redis-password='password'
```
```

4. ****Write Training Script** (`train\_ray.py`):**

```
```python
import ray
import torch
import torch.nn as nn
import torch.optim as optim
from ray import tune

Model
class SimpleNN(nn.Module):
 def __init__(self):
 super(SimpleNN, self).__init__()
 self.fc = nn.Linear(10, 1)
 self.sigmoid = nn.Sigmoid()

 def forward(self, x):
 return self.sigmoid(self.fc(x))

Training function
def train_model(config):
 model = SimpleNN()
 optimizer = optim.SGD(model.parameters(), lr=config["lr"])
 criterion = nn.BCELoss()

 # Dummy data
 data = torch.randn(1000, 10)
 labels = torch.randint(0, 2, (1000, 1)).float()

 for epoch in range(10):
 optimizer.zero_grad()
 output = model(data).squeeze()
 loss = criterion(output, labels)
 loss.backward()
 optimizer.step()
 tune.report(loss=loss.item())

Run distributed training
```

```
ray.init(address="auto", _redis_password="password")
tune.run(
 train_model,
 config={"lr": tune.grid_search([0.01, 0.1])},
 num_samples=2,
 resources_per_trial={"cpu": 2}
)
```

5. **\*\*Run the Script\*\*** (Laptop 1):

```
``bash
python3 train_ray.py
``
```

**\*\*Pros\*\*:**

- Simple setup and Python-native.
- Excellent for distributed ML and hyperparameter tuning.
- Lightweight and flexible.

**\*\*Cons\*\*:**

- Less mature for big data processing compared to Spark.
- Limited fault tolerance for large clusters.

**\*\*Use Case\*\*:** Ideal for distributed ML with PyTorch/TensorFlow or reinforcement learning.